Харківський національний університет радіоелектроніки
Факультет комп’ютерних наук
Кафедра програмної інженерії

# ЗВІТ

з лабораторної роботи № 3

з дисципліни «Програмування мовою Dart»

на тему: «Асинхронна та паралельна обробка даних: Future, Stream, ізоляти»

Розклад занять групи (варіант 4)

Виконала:
ст. гр. ПЗПІ-23-9
Курінна Д. С.

Перевірив:
асистент каф. ПІ
Баличев М. Д.

Харків 2026

1.1 Мета роботи та завдання

Мета роботи: набути практичних навичок асинхронного й паралельного програмування мовою Dart: `Future` та `async/await`, потоки `Stream`, ізоляти, а також написання модульних тестів.

Завдання за варіантом 4.

Предметна область, сутність і вхідні дані ті самі, що в ЛР 2: «Розклад занять групи», заняття з полями дисципліна, викладач, тип, день тижня, час початку, аудиторія та дата першого заняття. Модель із ЛР 2 доповнено асинхронним шаром, потоками подій та ізолятом. Джерелом даних слугує пам'ять із затримками `Future.delayed`: файли й мережа з'являться в ЛР 4.

Програма має:

1) увімкнути лінти `unawaited_futures` і `discarded_futures`;
2) мати асинхронне сховище з операціями `load`, `save`, `delete`;
3) показати помилки забутого `await` і неспійманого винятку в `Future`;
4) порівняти послідовне й одночасне виконання та обробку помилок `Future.wait` і `.wait`;
5) мати генератор `async*` і показати лінивість потоку;
6) мати джерело подій на `StreamController` та підписку через `listen` із явним `cancel`;
7) порівняти `await for` і `listen` з `cancelOnError: false` на потоці з помилкою;
8) показати різницю між потоком одноразової підписки та широкомовним;
9) побудувати конвеєр `where`, `map`, `take` над подіями;
10) винести важке обчислення в `Isolate.run`, виміряти час і показати ізоляцію глобальних змінних;
11) мати щонайменше 6 модульних тестів (`package:test`), зокрема на помилку й порядок подій потоку;
12) проходити `dart analyze`, `dart format` і `dart test` без зауважень.

## 1.2 Хід роботи

### 1.2.1 Структура програми

Роботу виконано на основі ЛР 2. Усі лабораторні лежать в одному репозиторії як гілки `lab1`, `lab2` і `lab3`; код цієї роботи в гілці `lab3`: https://github.com/NureKurinnaDaria/dart-lab1-schedule/tree/lab3. Порівняти реалізації можна командою `git diff lab2 lab3 --stat` у клоні репозиторію: історії гілок незалежні, тому веб-порівняння GitHub недоступне. Склад проєкту наведено в таблиці 1. Логіка лежить у `lib/`, у `bin/` лишається сценарій демонстрацій, а тести підключають код через `package:lab3_schedule/...`. Альтернатива, один файл у `bin/`, не дозволила б підключити код до тестів.

Нових залежностей часу виконання немає: використано стандартні `dart:async` та `dart:isolate`. Пакет `test`, що був у `dev_dependencies` ще з шаблону, у цій роботі використано вперше [7]. Роботу виконано на Dart SDK 3.13.2, код оформлено за посібником Effective Dart [8].

Таблиця 1 – Склад проєкту

| Файл | Призначення |
|---|---|
| `bin/lab3_schedule.dart` | Сценарій усіх демонстрацій |
| `lib/lesson.dart` | Модель ЛР 2 (виправлено `id` та конструктори) |
| `lib/async_repository.dart` | Асинхронне сховище, потік `watchAll` |
| `lib/schedule_events.dart` | Події розкладу й джерело подій |
| `lib/heavy.dart` | Важке обчислення для ізолята |
| `test/*.dart` | Чотири файли модульних тестів |

### 1.2.2 Лінти

До `analysis_options.yaml` додано правило `discarded_futures`; `unawaited_futures` було ввімкнено ще в ЛР 1 (лістинг 1). Перше ловить виклик асинхронної функції в синхронному коді без використання результату, друге ловить `Future` без `await` в `async`-функції. Код із ЛР 2 не містив жодного `Future`, тому лінти нічого не підсвітили. [6]

Лістинг 1 – Правила лінтера

```
linter:
  rules:
    - prefer_single_quotes
    - always_declare_return_types
    - prefer_final_locals
    - unawaited_futures
    - discarded_futures
```

### 1.2.3 Асинхронне сховище

`AsyncRepository<T extends Identifiable>` має ті самі обмеження типу, що й синхронне сховище з ЛР 2. Кожна операція чекає імітовану затримку через `Future.delayed` (лістинг 2). `load` повертає `null`, якщо елемента немає: це очікувана ситуація, як і невдалий розбір рядка в попередніх роботах, тому виняток не потрібен. Результат роботи показано на рисунку 1. [1]

Лістинг 2 – Асинхронне сховище

```
class AsyncRepository<T extends Identifiable> {
  final Map<String, T> _items = {};
  int get length => _items.length;
  Future<T?> load(String id) async {
    await Future<void>.delayed(loadDelay);
    return _items[id];
  }
  Future<void> save(T item) async {
    await Future<void>.delayed(saveDelay);
    _items[item.id] = item;
  }
  Future<bool> delete(String id) async {
    await Future<void>.delayed(deleteDelay);
    return _items.remove(id) != null;
  }
}
```

### 1.2.4 Помилки з Future

Без `await` виклик `save` повертає незавершений `Future`, тому запис у сховище ще не відбувся й `length` дорівнює 0; з `await` дорівнює 1 (лістинг 3, рисунок 1). Лінт `unawaited_futures` попереджає про це (рисунок 2). Коментар `ignore` у демонстрації вимикає його на один рядок свідомо, щоб показати поведінку. [1]

Лістинг 3 – Забутий `await`

```
// ignore: unawaited_futures
repository.save(lessons.first);
stdout.writeln('Без await: ${repository.length} занять');
await repository.save(lessons.first);
stdout.writeln('З await: ${repository.length} занять');
```

Помилка асинхронної операції без `await` не потрапляє в `catch`: блок `try` завершується за мілісекунди, а виняток виникає через 500 мс, коли блоку вже немає в стеку викликів. Тому він потрапляє в обробник неспійманих помилок зони `runZonedGuarded`; без нього така помилка завершила б програму. З `await` виняток кидається всередині `try`, і `catch` його ловить (лістинг 4, рисунок 3. [2]

Лістинг 4 – Неспіймана помилка у зоні

```
await runZonedGuarded(
  () async {
    try {
      // ignore: unawaited_futures
      failLater();
      stdout.writeln('  try завершився без помилки');
    } on Exception catch (error) {
      stdout.writeln('  catch спіймав: $error');
    }
    await Future<void>.delayed(operationDelay * 2);
  },
  (error, stack) {
    stdout.writeln('  потрапила в зону: $error');
  },
);
```

1.2.5 Послідовне й одночасне виконання

Три операції по 500 мс виконано послідовно та через `Future.wait` (лістинг 5): отримано 1541 мс і 514 мс (рисунок 4). Асинхронність не прискорює обчислення, а прибирає простій під час очікування: три затримки йдуть одночасно.

Лістинг 5 – Заміри часу

```
final sequential = Stopwatch()..start();
await operation();
await operation();
await operation();
sequential.stop();
final concurrent = Stopwatch()..start();
await Future.wait([operation(), operation(), operation()]);
concurrent.stop();
```

Для обробки помилки порівняно `Future.wait` і розширення `.wait` (лістинг 6): з трьох одночасних операцій друга завершується помилкою. `Future.wait` повідомляє лише про цю помилку, а результати двох успішних операцій втрачає. `.wait` кидає `ParallelWaitError`, у якому в `values` лежать результати успішних операцій (`[1, null, 3]`), а в `errors` помилка з її позицією (`[null, Exception: …, null]`). Тож `.wait` показує, яка саме операція впала і що повернули решта (рисунок 4.

Лістинг 6 – `Future.wait` і `.wait`

```
try {
  await Future.wait([succeed(1), fail(2), succeed(3)]);
} on Exception catch (error) {
  stdout.writeln('  помилка: $error');
}
try {
  await [succeed(1), fail(2), succeed(3)].wait;
} on ParallelWaitError<List<int?>, List<AsyncError?>> catch (error) {
  stdout.writeln('  значення: ${error.values}');
}
```

### 1.2.6 Генератор і лінивість потоку

Метод `watchAll` оголошено як `async*`: він видає елементи по одному через `yield` із затримкою (лістинг 7). Для перевірки лінивості створено генератор, який друкує рядок на початку тіла. Рядок з'явився лише після підписки, а не при виклику функції (рисунок 5): виклик лише створює потік, а тіло `async*` починає виконуватись, коли на потік підписуються. [4]

Лістинг 7 – Генератор `async*`

```
Stream<T> watchAll() async* {
  for (final item in _items.values.toList()) {
    await Future<void>.delayed(watchDelay);
    yield item;
  }
}
```

### 1.2.7 Джерело подій на StreamController

Події розкладу описано запечатаним класом `ScheduleEvent` (підтипи `LessonAdded` і `LessonRemoved`), а `ScheduleEventSource` будує потік на `StreamController` (лістинг 8). Три події додано до підписки: контролер з одноразовою підпискою буферизує їх і віддає слухачу в тому самому порядку, тому вони не загубилися. Підписку збережено й явно скасовано через `cancel()`; подія, додана після скасування, не дійшла (лістинг 9, рисунок 6). [3]

Лістинг 8 – Події й джерело подій

```
sealed class ScheduleEvent {
  const ScheduleEvent();
}
class LessonAdded extends ScheduleEvent {
  const LessonAdded(this.lesson);
  final Lesson lesson;
}
class ScheduleEventSource {
  final StreamController<ScheduleEvent> _controller =
```

```
      StreamController<ScheduleEvent>();
  Stream<ScheduleEvent> get events => _controller.stream;
  void add(ScheduleEvent event) => _controller.add(event);
  Future<void> close() => _controller.close();
}
```

Лістинг 9 – Підписка з явним `cancel`

```
final subscription = source.events.listen(received.add);
await Future<void>.delayed(Duration.zero);
// ... три події отримано
await subscription.cancel();
source.add(LessonRemoved(lessons.first)); // не дійде
```

1.2.8 Помилки в потоці, типи підписки й конвеєр

Один і той самий потік 1, 2, помилка, 3, 4 прочитано двома способами (лістинг 10, рисунок 7). `await for` перетворює помилку на виняток, цикл переривається, і значення 3 і 4 не отримано. `listen` із `cancelOnError: false` передає помилку в `onError` і продовжує, тому дійшли й значення 3 і 4. Потік з одноразовою підпискою кидає `StateError` на другу підписку з текстом `Bad state: Stream has already been listened to.`, а широкомовний віддає подію всім слухачам (лістинг 11). Кожну підписку збережено в змінній і скасовано через `cancel()`, зокрема й невдалу другу (`second?.cancel()`). [3]

Лістинг 10 – `await for` і `listen` з помилкою

```
Future<Stream<int>> _numbersWithError() async {
  final controller = StreamController<int>()
    ..add(1)
    ..add(2)
    ..addError(Exception('помилка посеред потоку'))
    ..add(3)
    ..add(4);
  unawaited(controller.close());
  return controller.stream;
}
try {
  await for (final value in await _numbersWithError()) {
    stdout.writeln('  отримано: $value');
  }
} on Exception catch (error) {
  stdout.writeln('  потік перервано: $error');
}
final subscription = (await _numbersWithError()).listen(
  (value) => stdout.writeln('  отримано: $value'),
  onError: (Object error) => stdout.writeln('  помилка: $error'),
  onDone: done.complete,
  cancelOnError: false,
);
```

Лістинг 11 – Одноразова й широкомовна підписка

```
final single = StreamController<int>();
final first = single.stream.listen(
  (value) => stdout.writeln('  1: $value'),
);
StreamSubscription<int>? second;
try {
  second = single.stream.listen(
    (value) => stdout.writeln('  2: $value'),
  );
} on StateError catch (error) {
  stdout.writeln('  друга підписка: $error');
}
await first.cancel();
await second?.cancel();
```

Конвеєр `where` → `cast` → `map` → `take` над подіями винесено у функцію `addedLabels` у `lib/`, щоб його можна було перевірити тестом (лістинг 12). `where` відфільтровує `LessonRemoved`, `cast<LessonAdded>()` звужує тип елементів, `map` перетворює події на рядки, а `take` обриває потік після трьох елементів. Методу `whereType` у `Stream`, на відміну від `Iterable`, немає, тому використано `where` із `cast`: приведення безпечне, бо після фільтра інших подій у потоці не лишається.

Лістинг 12 – Конвеєр над потоком подій

```
Stream<String> addedLabels(Stream<ScheduleEvent> events, {required int
count}) {
  return events
      .where((event) => event is LessonAdded)
      .cast<LessonAdded>()
      .map((event) => 'нове заняття: ${event.lesson}')
      .take(count);
}
```

### 1.2.9 Ізоляти

Важке синхронне обчислення `countConflicts` (повний перебір пар слотів) виконано в основному ізоляті й через `Isolate.run` (лістинг 13). Результат однаковий, час приблизно рівний: 1096 мс і 928 мс. Різниця в часі мала й може бути випадковою, тому не можна стверджувати, що ізолят швидший. Суттєва різниця в іншому: поки обчислення йде в основному ізоляті, таймер із періодом 10 мс не спрацював жодного разу (0 тактів), а в `Isolate.run` спрацював 63 рази, бо основний ізолят лишався вільним (рисунок 8). [5]

Лістинг 13 – Важке обчислення в `Isolate.run`

```
int countConflicts(int slots) {
  var conflicts = 0;
  for (var i = 0; i < slots; i++) {
    for (var j = i + 1; j < slots; j++) {
      final weighted = i * firstSlotWeight + j * secondSlotWeight;
      if (weighted % conflictModulus == 0) conflicts++;
    }
  }
  return conflicts;
}
final result = await Isolate.run(() => countConflicts(slotCount));
```

Спроба змінити глобальну змінну всередині `Isolate.run` показала, що кожен ізолят має власну пам’ять: усередині ізолята було 0 (а не 1) і стало 100, а в основному ізоляті значення лишилось 1 (лістинг 14). Нова копія ізолята ініціалізує глобальні змінні заново, а між ізолятами передаються лише повідомлення. [5]

Лістинг 14 – Глобальна змінна в ізоляті

```
int globalCounter = 0;
globalCounter = initialCounter;
final (before, after) = await Isolate.run(() {
  final before = globalCounter;
  globalCounter = isolateCounter;
  return (before, globalCounter);
});
```

1.2.10 Модульні тести

Написано 15 тестів у чотирьох файлах (лістинг 15). У `async_repository_test.dart` перевіряються збереження й завантаження, відсутній ідентифікатор, видалення, те, що заняття з різними викладачами мають різні `id` і обидва лишаються у сховищі, та порядок елементів потоку `watchAll`. У `schedule_events_test.dart` перевіряються порядок подій, збереження подій, доданих до підписки, фільтрація й обрізання потоку в `addedLabels` та проходження помилки джерела крізь конвеєр після значень. У `heavy_test.dart` перевіряються `countConflicts` для порожнього й малого входу, відоме значення для 98 слотів і збіг результату в основному ізоляті та в `Isolate.run`. У `lesson_test.dart` перевіряються кеш `fromCsv`, `FormatException` для некоректного рядка та рівність об’єктів. Використано `group`, `setUp` і `expectLater` [7]; є тести на помилку й на порядок подій (рисунок 9).

Лістинг 15 – Фрагмент тестів

```
group('AsyncRepository', () {
  test('save і load повертають збережене заняття', () async {
    await repository.save(first);
    expect(await repository.load(first.id), first);
  });
});
await expectLater(
  repository.watchAll(),
  emitsInOrder([first, second, emitsDone]),
);
```

1.2.11 Зауваження до ЛР 2

За відгуком викладача до ЛР 2 виконано три виправлення. `Lesson.id` складався з п’яти полів, а == і `hashCode` порівнюють сім, тому сховище вважало двох викладачів одного слоту одним записом; тепер `id` містить `teacher` і `firstDate`. Усі чотири підтипи `Lesson` оголошено первинним конструктором. Роботи зведено в один репозиторій із гілками `lab1`, `lab2`, `lab3`.

1.3 Результати виконання

Підсумки експериментів наведено в таблиці 2, вивід програми показано на рисунках 1–8, результат перевірок коду та тестів на рисунку 9.

Таблиця 2 – Результати експериментів

| Пункт | Експеримент | Результат |
|---|---|---|
| Забутий `await` | Довжина сховища | Без `await` 0, з `await` 1 |
| Час | Послідовно / одночасно | 1541 мс / 514 мс |
| Помилка `Future` | `Future.wait` / `.wait` | Лише помилка / `[1, null, 3]` і помилка з позицією |
| Неспіймана помилка | Без `await` / з `await` | Потрапила в зону / спіймав `catch` |
| Лінивість | Виклик генератора | Тіло виконується лише після підписки |
| Скасування | Подія після `cancel` | Отримано 3 події з 4 |
| Помилка в потоці | `await for` / `listen` | 1, 2, переривання / 1, 2, помилка, 3, 4 |
| Друга підписка | Одноразовий потік | `Bad state: Stream has already been listened to.` |
| Ізолят | Час і такти таймера | 1096 мс, 0 / 928 мс, 63 |
| Глобальна змінна | В ізоляті / в основному | 0 → 100 / 1 |
| Тести | `dart test` | 15 тестів, All tests passed! |

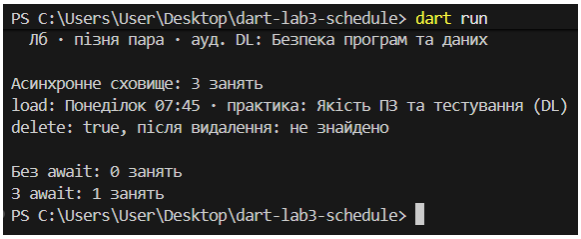



Рисунок 1 – Асинхронне сховище і забутий await: сховище до й після очікування

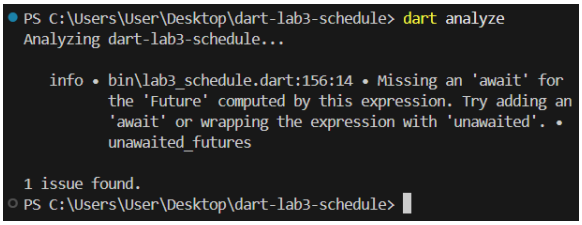



Рисунок 2 – Попередження лінта `unawaited_futures`

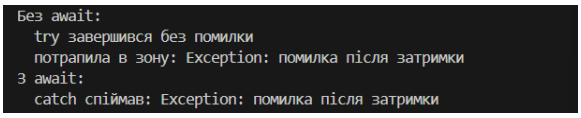



Рисунок 3 – Помилка без `await` і з `await`

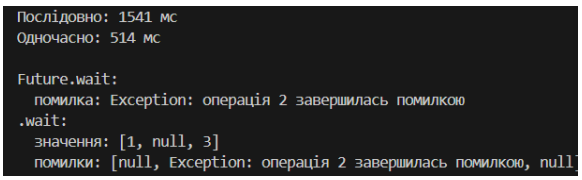



Рисунок 4 – Послідовне й одночасне виконання, `Future.wait` і `.wait`

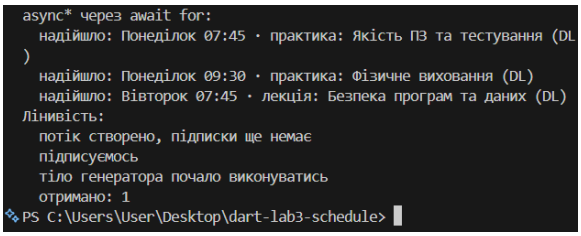



Рисунок 5 – Генератор `async*` і лінивість

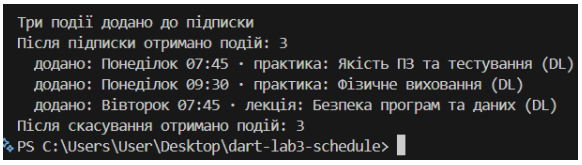



Рисунок 6 – `StreamController`: події до підписки й після скасування

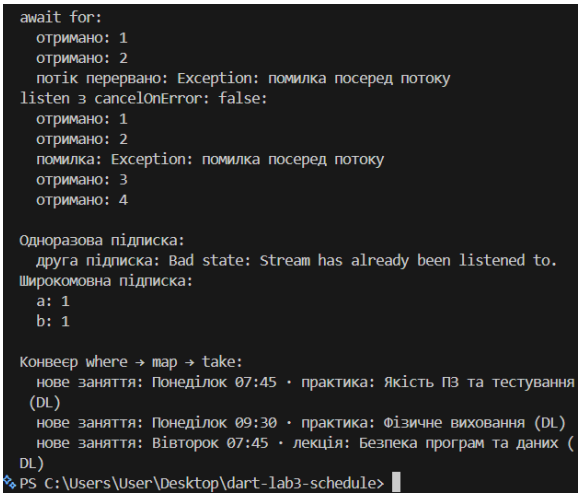



Рисунок 7 – Помилка в потоці, типи підписки, конвеєр

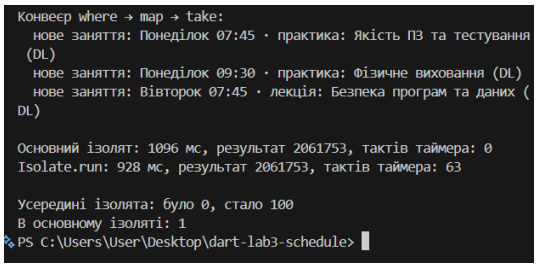



Рисунок 8 – Ізолят: час, такти таймера, глобальна змінна

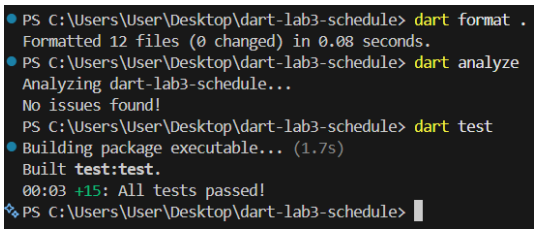



Рисунок 9 – Результат `dart format`, `dart analyze` і `dart test`

## 1.4 Висновки

У роботі модель розкладу занять з ЛР 2 доповнено асинхронним шаром. Реалізовано асинхронне сховище, генератор `async*`, джерело подій на `StreamController`, конвеєр над потоком, обчислення в `Isolate.run`, а також 15

модульних тестів. Експерименти показали, що забутий `await` залишає результат нерозрахованим, а помилка без `await` не потрапляє в `try/catch`. Одночасне виконання трьох очікувань триває приблизно стільки ж, скільки одне, а `.wait` на відміну від `Future.wait` зберігає результати успішних операцій і позицію помилки.

Потік з одноразовою підпискою буферизує події до першої підписки, а широкомовний віддає подію всім слухачам. `await for` завершується на першій помилці, а `listen` із `cancelOnError: false` продовжує. Ізолят не блокує основний цикл подій, а кожен ізолят має власні глобальні змінні. Код проходить `dart analyze` без зауважень, `dart format` не змінює файлів, `dart test` проходить. Історія гілки `lab3` налічує понад п’ять комітів. Вимірювані часи залежать від комп’ютера.

1.5 Перелік використаних джерел

1. Dart Team. Asynchronous programming: futures, async, await. URL: https://dart.dev/libraries/async/async-await (дата звернення: 06.11.2026-09.11.2026).

2. Dart Team. Futures and error handling. URL: https://dart.dev/libraries/async/futures-error-handling (дата звернення: 06.11.2026-09.11.2026).

3. Dart Team. Asynchronous programming: Streams. URL: https://dart.dev/libraries/async/using-streams (дата звернення: 06.11.2026-09.11.2026).

4. Dart Team. Creating streams in Dart. URL: https://dart.dev/libraries/async/creating-streams (дата звернення: 06.11.2026-09.11.2026).

5. Dart Team. Concurrency in Dart. URL: https://dart.dev/language/concurrency (дата звернення: 06.11.2026-09.11.2026).

6. Dart Team. Linter rules. URL: https://dart.dev/tools/linter-rules (дата звернення: 06.11.2026-09.11.2026).

7. Dart Team. package:test. URL: https://pub.dev/packages/test (дата звернення: 06.11.2026-09.11.2026).

8. Dart Team. Effective Dart. URL: https://dart.dev/effective-dart (дата звернення: 06.11.2026-09.11.2026).